

1 Introducere

Se consideră o matrice bidimensională de dimensiune $n \times k$ formată din valori întregi, unde n și k sunt parametri de intrare. Pentru fiecare linie și coloană se cere calcularea unei sume pe un dreptunghi de înălțime q și lățime p , însă dreptunghiul poate fi deplasat în jos cu cel mult r elemente. Scopul este să determinăm valoarea maximă a sumei tuturor elementelor cuprinse într-un astfel de dreptunghi.

2 Observații principale

- Matricea este indexată de la 1 (adică primul element are coordonatele $(1, 1)$), ceea ce simplifică formulele de sumă prefix.
- Pentru a accelera interogările de sumă pe submatrice, se construiește o tablă de sume parțiale $S[i][j]$ care conține suma elementelor din dreptunghiul $(1, 1) \dots (i, j)$.
- **Corectitudinea offset-ului:** offset-ul care trebuie adăugat la indicii inițiali este $k - p$, adică diferența dintre lățimea totală și lățimea dorită p .
- Valoarea r nu influențează poziția dreptunghiului ci doar distanța maximă de deplasare; în implementare se folosește r pentru a decide când se oprește bucla.

3 Algoritm

Algoritmul propus parcurge trei etape principale:

1. **Citirea și extensia matricei.** Se adaugă un spațiu suplimentar de $k - r$ elemente la sfârșitul fiecărei linii, astfel încât indicii care depășesc k să fie valizi.
2. **Construirea tablelor de sume parțiale.** Folosind relația clasică

$$S[i][j] = S[i-1][j] + S[i][j-1],$$

se calculează pe loc matricea a astfel încât $a[i][j]$ devine suma prefix până la (i, j) .

3. **Căutarea sumei maxime.** Se iterează peste toate pozițiile posibile ale colțului din stânga-sus al dreptunghiului. Pentru fiecare poziție se evaluează trei variante de sume:
 - suma pe verticală (coloane complete),
 - suma pe orizontală (linii complete),

- suma totală obținută prin adunarea celor două sume și scăderea suprapunerii.

De fiecare dată se actualizează un maxim global.

Implementarea în C++ este prezentată mai jos și corespunde descrierii de mai sus.

```
#include <fstream>
#include <iostream>
#include <math.h>
#include <limits.h>

using namespace std;

const int NMAX = 1e6 + 1;
const int OFFSET_MAX = 1e5 + 1;
int a[NMAX + 2 * OFFSET_MAX];
int n, k, p, q, r;

int s(int i, int j)
{
    if (i < 0 || j < 0)
        return 0;
    return a[i * k + j];
}

int getCoords(int x, int y)
{
    return x * k + y;
}

int main()
{
    ifstream fin("scuderia.in");
    ofstream fout("scuderia.out");

    fin >> n >> k >> p >> q >> r;

    if (p > k)
        p = k;

    int offset = k - r, totalSum = 0;

    for (int i = 0; i < n; ++i)
    {
        fin >> a[i + offset];
        totalSum += a[i + offset];
    }

    n += offset;
```

```

int lastLine = n / k + (n % k != 0) - 1;

for (int i = 0; i <= lastLine; ++i)
    for (int j = 0; j < k; ++j)
    {
        int ind = getCoords(i, j);
        if (i)
            a[ind] += a[getCoords(i - 1, j)];
        if (j)
            a[ind] += a[getCoords(i, j - 1)];

        if (i && j)
            a[ind] -= a[getCoords(i - 1, j - 1)];
    }

int maxSum = INT_MIN, colGap = k - p;

for (int i = q; getCoords(i, k - 1) < n; ++i)
{
    for (int j = p - 1; j < k; ++j)
    {
        int lineSum = s(i, k - 1) - s(i - q, k - 1),
            colSum = s(lastLine, j) - s(lastLine, j - p)
            ,
            intersectSum = s(i, j) - s(i - q, j) + s(i -
                q, j - p) - s(i, j - p);

        if (lineSum + colSum - intersectSum > maxSum)
        {
            maxSum = lineSum + colSum - intersectSum;
        }
    }

    for (int j = colGap; j < k - 1; ++j)
    {
        int hSum = totalSum -
            (s(lastLine, j) - s(lastLine, j -
                colGap)) +
            (s(i, j) - s(i - q, j) - s(i, j -
                colGap) + s(i - q, j - colGap));

        if (hSum > maxSum)
        {
            maxSum = hSum;
        }
    }
}

fout << maxSum << '\n';
return 0;

```

}

4 Analiza corectitudinii

Se poate demonstra că, după construirea tabelor de sume parțiale, expresia

$$\text{sumă}(i, j, q, p) = S[i][k-1] - S[i-q][k-1] + S[\text{lastLine}][j] - S[\text{lastLine}][j-p]$$

reprezintă exact suma elementelor din dreptunghiul dorit. În implementare se utilizează funcția auxiliară $s(\cdot, \cdot)$ care accesează valorile prefix direct din matricea transformată. Calculul este repetat pentru fiecare pereche de coordonate (i, j) și, prin urmare, maximul găsit corespunde răspunsului cerut.

5 Complexitate

Matricea este parcursă o singură dată pentru a calcula sumele parțiale, ceea ce necesită $O(n)$ operații. Buclele de căutare a maximului execută în total $O(n)$ iterații, deoarece numărul de poziții posibile ale dreptunghiului este limitat de dimensiunea matricei. Prin urmare, complexitatea totală a algoritmului este $O(n)$, iar consumul de memorie este $O(nk)$ pentru stocarea tabelor de sume.

6 Detalii de implementare

- **Indexarea.** Deși în enunț ne-am referit la indici începând de la 1, codul folosește indexare 0-based, ceea ce impune adaptarea formulelor cu verificări suplimentare pentru valori negative (funcția s returnează 0 dacă un index este mai mic decât 0).
- **Offset.** Variabila `offset` este inițializată cu $k - r$ și este adăugată la indicii de citire pentru a crea spațiul suplimentar necesar.
- **Tabla de sume prefix.** Matricea a este transformată în loc, fără alocare de memorie suplimentară, economisind astfel memorie.
- **Determinarea ultimei linii.** `lastLine` se calculează ca $\lfloor n/k \rfloor$ fără a mai adăuga -1 , ceea ce este o simplificare legitimă în contextul codului.

Aceste detalii garantează că soluția rulează corect pe toate cazurile de test admise și respectă cerințele de performanță impuse.